Applications

Applications-2

6. **Check if the given graph G is strongly connected.**
7. Given a directed graph G, output all the strongly connected components.
8. Given a graph G, check if G is 2-edge-connected.
9. Given a graph G, check if G is 2-vertex-connected.
10. Check if a graph G is planar.

Connected graphs in directed setting

Strongly connected: For every pair of vertices u,v

there is a directed path from u to v, AND from v to u

Unilaterally connected: For every pair of vertices u,v

there is a directed path from u to v, OR from v to u

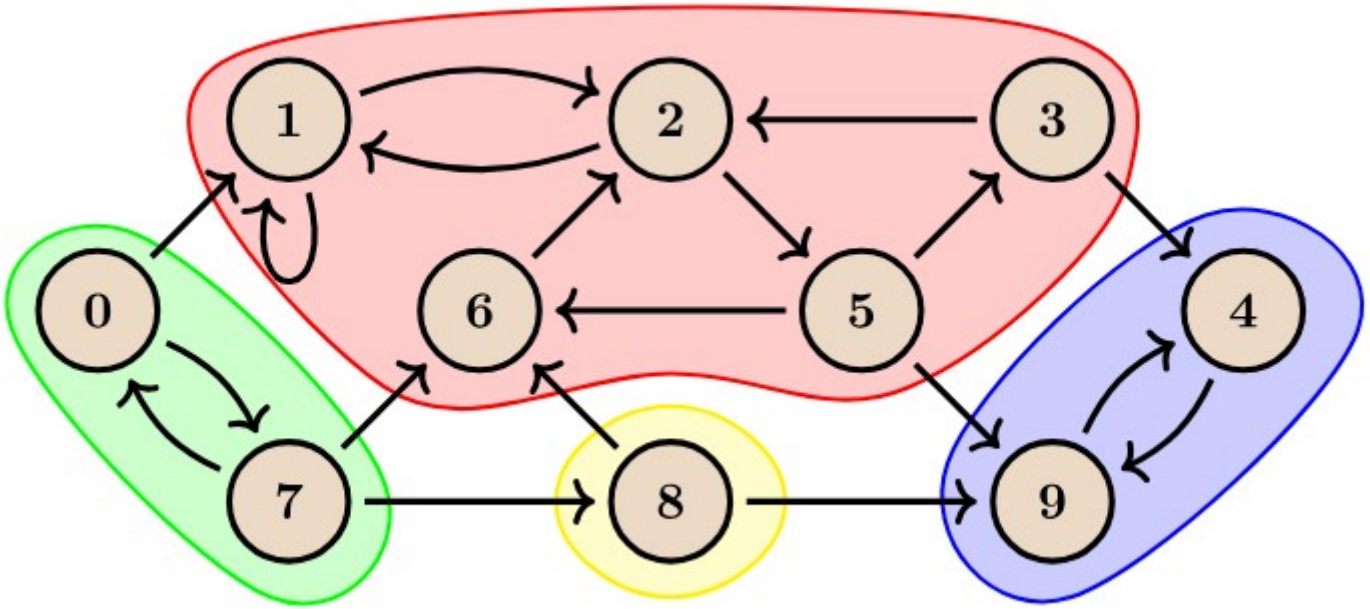
Weakly connected: For every pair of vertices u,v

there is a path between u and v, (in the underlying undirected setting)

Strongly connected

A graph G is **strongly connected**, if there exists a path from u to v, and from v to u, for all pairs u,v.

Solution-1:



Strongly connected

A graph G is **strongly connected**, if there exists a path from u to v, and from v to u, for all pairs u,v.

Solution-1: Run DFS on each vertex, and check if every other vertex is visited. Time: O(V(V+E))

Solution-2:

Strongly connected

Solution-1: Run DFS on each vertex, and check if every other vertex is visited. Time: O(V(V+E))

Solution-2:

DFS(G, v), for some v. If every other vertex is visited, go to the next step, otherwise “say NO”.

Consider a vertex w. What happens if there exists a directed path from w to v?

Strongly connected

Solution-1: Run DFS on each vertex, and check if every other vertex is visited. Time: O(V(V+E))

Solution-2:

DFS(G, v), for some v. If every other vertex is visited, go to the next step, otherwise “say NO”.

Consider a vertex w. What happens if there exists a directed path from w to v?

There is a directed path from w to every other vertex in G
+
There is a directed path from v to w

Strongly connected

Solution-1: Run DFS on each vertex, and check if every other vertex is visited. Time: O(V(V+E))

Solution-2:

DFS(G, v), for some v. If every other vertex is visited, go to the next step, otherwise “say NO”.

Consider a vertex w. What happens if there exists a directed path from w to v?

There is a directed path from w to every other vertex in G
+
There is a directed path from v to w

What happens if there is no directed path from w to v?

Strongly connected

Solution-2:

DFS(G, v), for some v. If every other vertex is visited, go to the next step, otherwise “say NO”.

Consider a vertex w. What happens if there exists a directed path from w to v?

What do you want now?

Suppose there is a directed path from every vertex in G to v?

How do I check if such a condition is true?

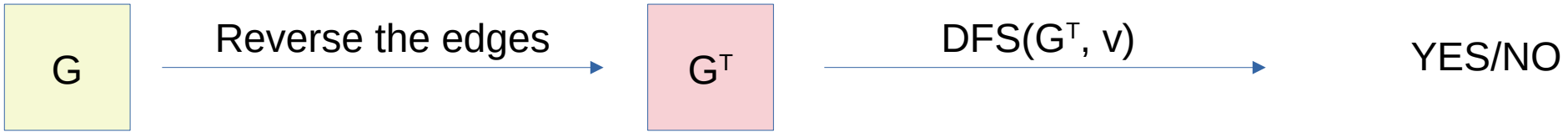
Strongly connected

Solution-2:

DFS(G, v), for some v. If every other vertex is visited, go to the next step, otherwise “say NO”.

Consider a vertex w. What happens if there exists a directed path from w to v?

What do you want now?

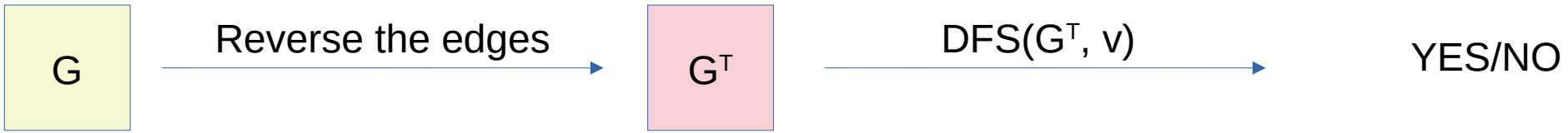


Strongly connected

Solution-2: analysis

2 DFS's + time taken to reverse the edges.

O(m+n) time.



Strongly connected

Solution-3: Use only one DFS.

Strongly connected

Solution-3: Use only one DFS.

Consider a subtree rooted at v, call it T_v.

1. What if there is no edge going outside T_v ? **What kind of edges?**

Strongly connected

Solution-3: Use only one DFS.

Consider a subtree rooted at v, call it T_v.

1. What if there is no edge going outside T_v ? **What kind of edges?**
2. Is it **necessary** and **sufficient**?
3. Sufficiency! Consider a vertex w, and use Step 1.

Edge going out of a subtree

Strongly connected

Solution-3: Use only one DFS.

Consider a subtree rooted at v, call it T_v.

1. What if there is no edge going outside T_v ? **What kind of edges?**
2. Is it necessary and sufficient?
3. Sufficiency! Consider a vertex w, and use Step 1.

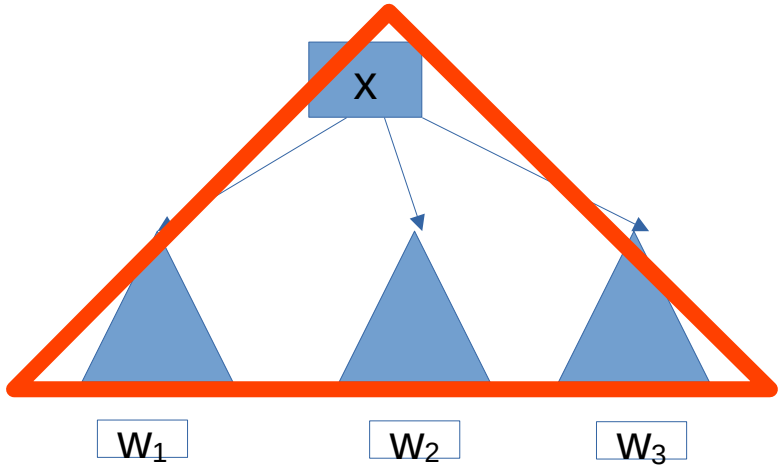
Edge going out of a subtree

How do I check if such a condition is true?

Strongly connected

Solution-3: Use only one DFS.

Return the vertex (or discovery time) with the **least discovery time** to which there is an edge from T_v.

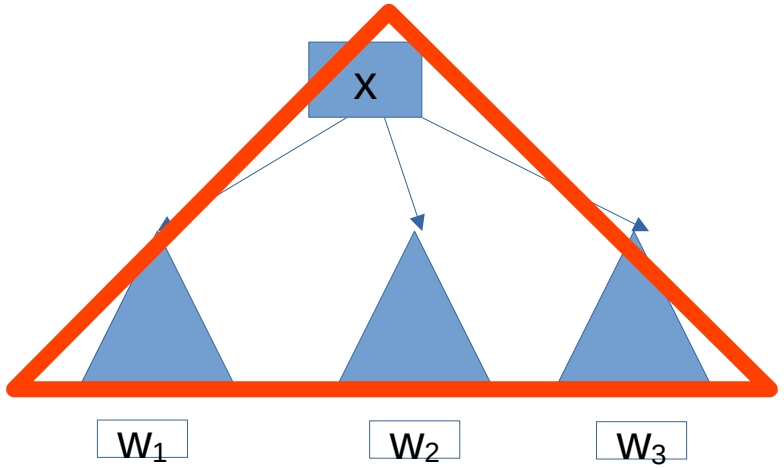


Strongly connected

What will you return?
What kind of edges are possible here?

Solution-3: Use only one DFS.

Return the vertex (or discovery time) with the **least discovery time** to which there is an edge from T_v.



PSEUDOCODE (continued)

```
DFS-VISIT(G, u)
 1   time = time + 1
 2   u.d = time
 3   u.color = GRAY
 4   for each vertex v in G.Adj[u]
 5         if v.color == WHITE
 6               v.π = u
 7               DFS-VISIT(G, v)
 8   time = time + 1
 9   u.f = time
10   u.color = BLACK
```

Applications-2

5. Check if the given graph G is strongly connected.

6. **Given a directed graph G, output all the strongly connected components.**

7. Given a graph G, check if G is 2-edge-connected.

8. Given a graph G, check if G is 2-vertex-connected.

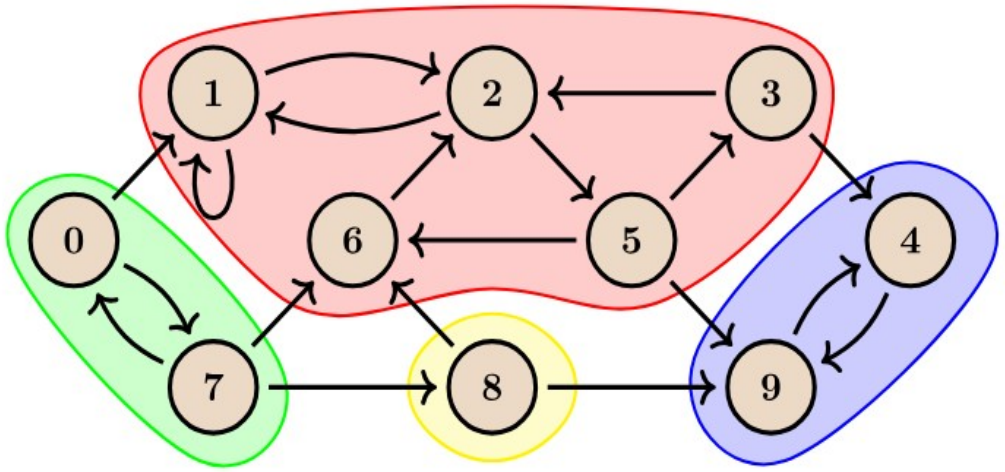
9. Check if a graph G is planar.

Strongly connected components

A **strongly connected component** (SCC) of G is a maximal set of vertices C (subset of V(G)) such that for all u, v in C ,

there is a directed path from u to v, and

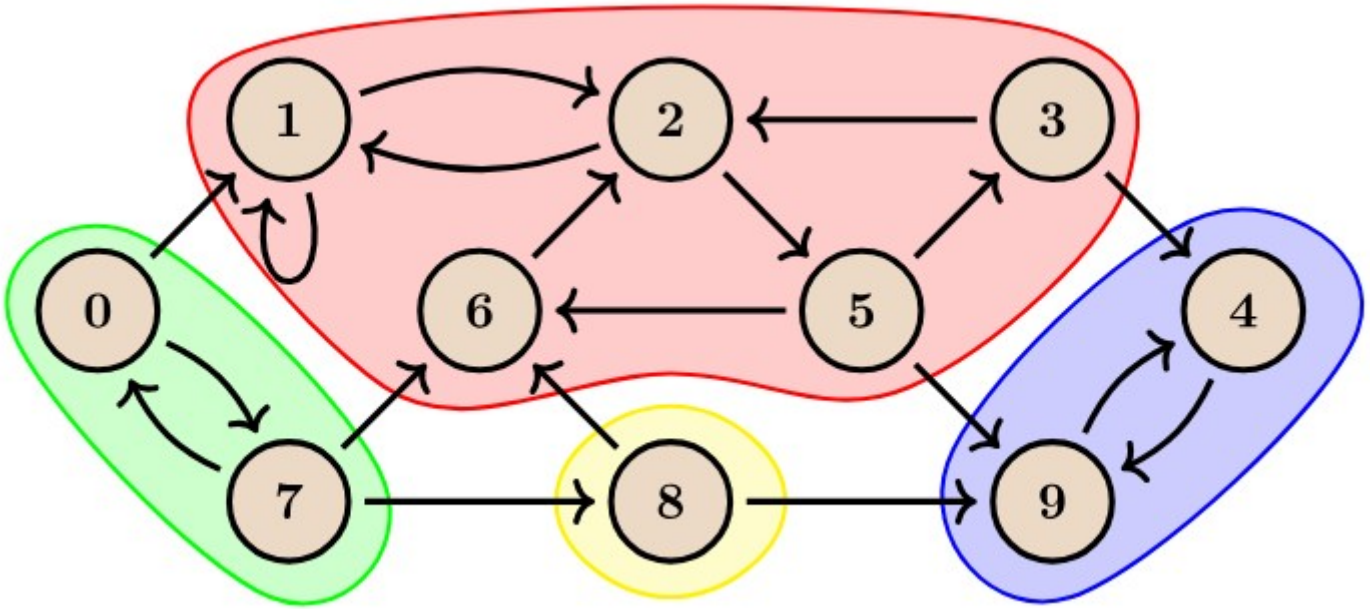
there is a directed path from v to u.



Strongly connected components: **Intuition**

Look at the last DFS-tree. What kind of outgoing edges are possible?

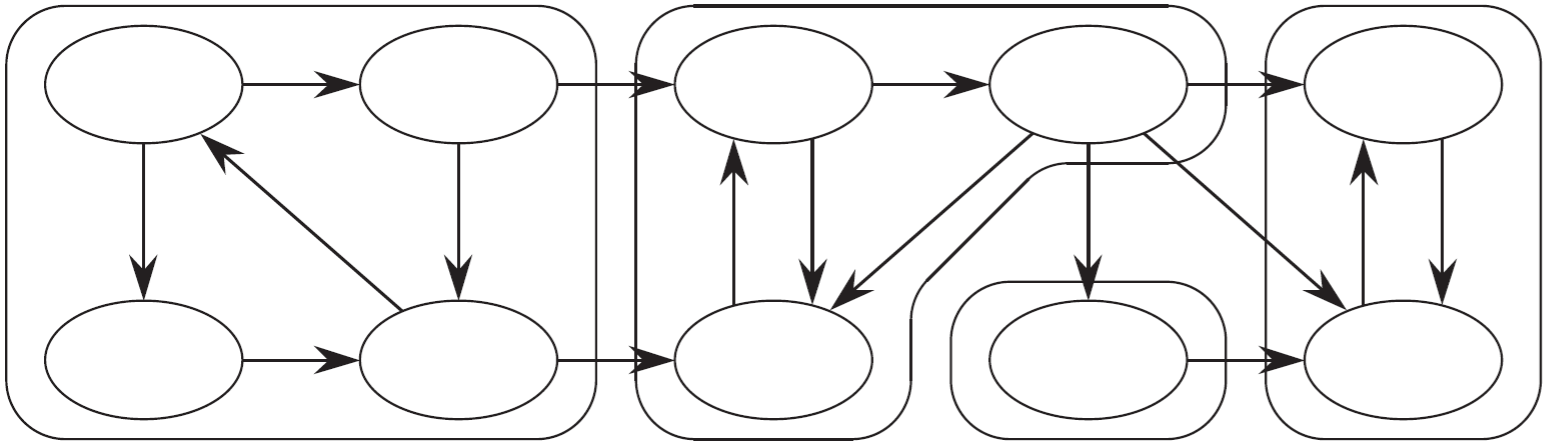
Decide on the SCC of the root of the DFS-tree.



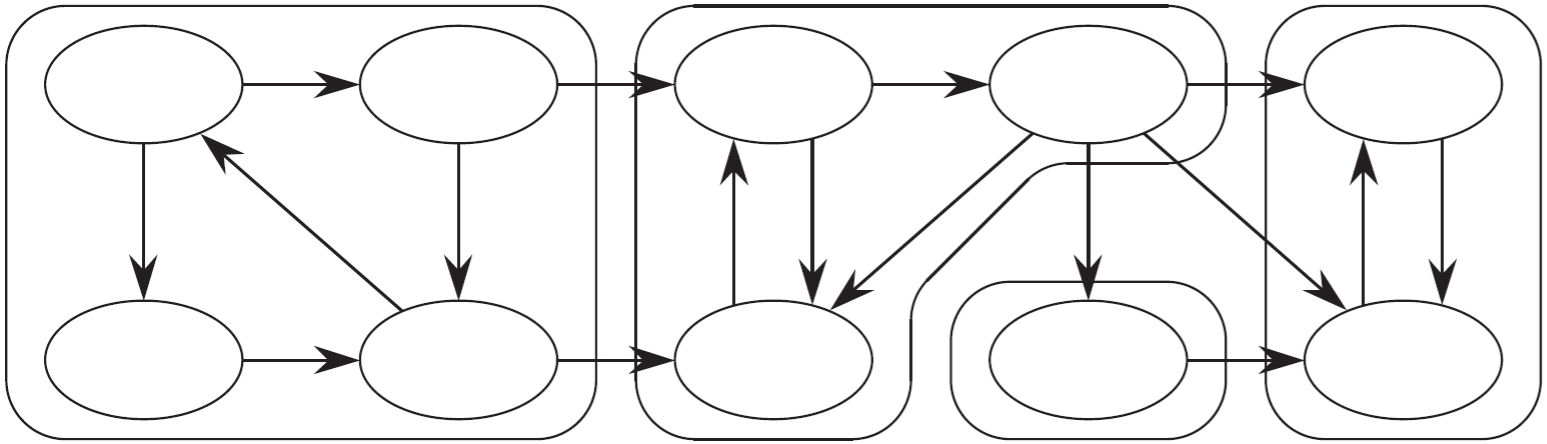
STRONGLY CONNECTED COMPONENT

STRONGLY-CONNECTED-COMPONENTS(G)

1 call DFS(G) to compute finish times $u.f$ for each vertex u
2 create G^T
3 call DFS(G^T), but in the main loop of DFS, consider the vertices
in order of decreasing $u.f$ (as computed in line 1)
4 output the vertices of each tree in the depth-first forest formed in line 3 as a
separate strongly connected component

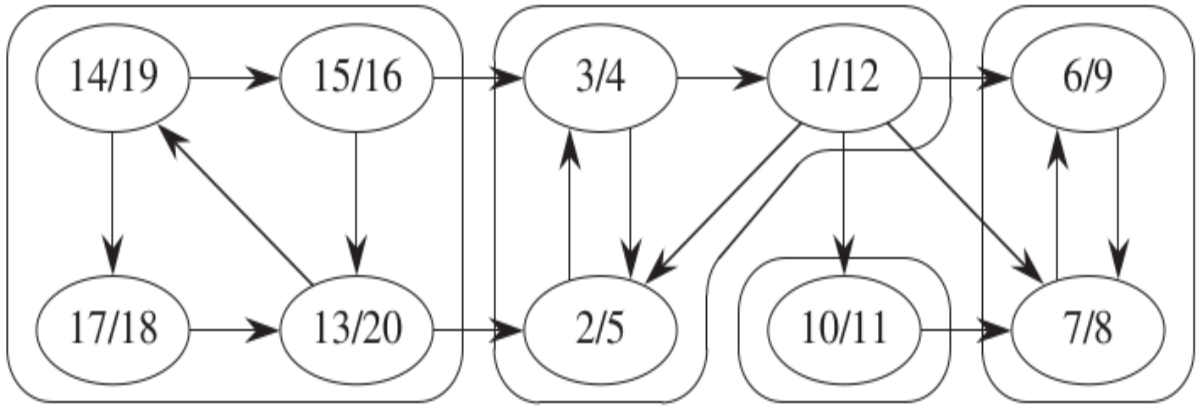


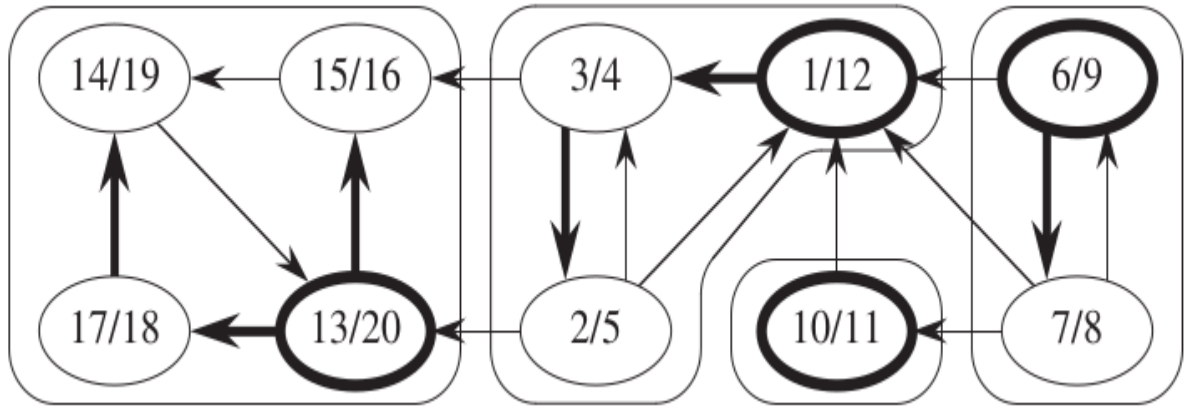
STRONGLY CONNECTED COMPONENT



STRONGLY-CONNECTED-COMPONENTS(G)

1 call DFS(G) to compute finish times $u.f$ for each vertex u
2 create G^T
3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component





EXAMPLE (continued)

Algorithm uses $G^{\mathrm{T}} =$ ***transpose*** of G.

- $G^{\mathrm{T}} = (V, E^{\mathrm{T}})$, $E^{\mathrm{T}} = \{(u, v) : (v, u) \in E\}$.
- G^{T} is G with all edges reversed.

Can create G^{T} in $\Theta(V + E)$ time if using adjacency lists.

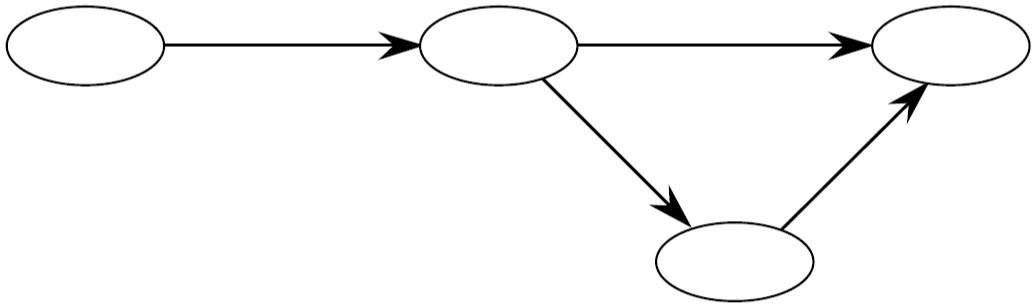
Observation

G and G^{T} have the *same* SCC's. (u and v are reachable from each other in G if and only if reachable from each other in G^{T}.)

EXAMPLE (continued)

- $G^{SCC} = (V^{SCC}, E^{SCC})$.
- V^{SCC} has one vertex for each SCC in G.
- E^{SCC} has an edge if there's an edge between the corresponding SCC's in G.

For our example:



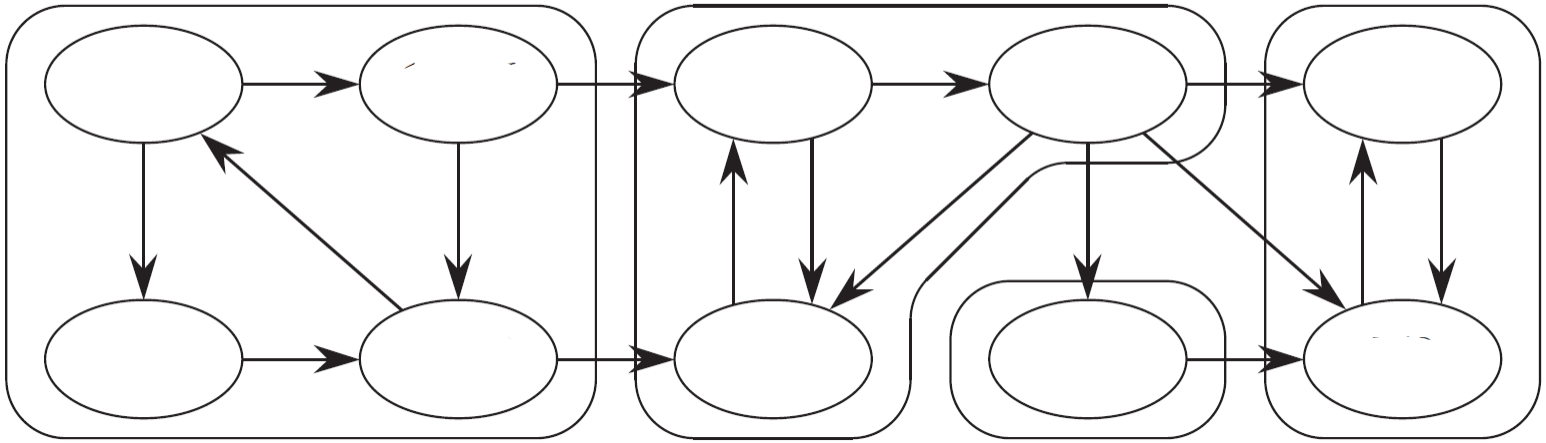
Lemma

G^{SCC} is a dag. More formally, let C and C' be distinct SCC's in G, let $u, v \in C$, $u', v' \in C'$, and suppose there is a path $u \rightsquigarrow u'$ in G. Then there cannot also be a path $v' \rightsquigarrow v$ in G.

EXAMPLE (continued)

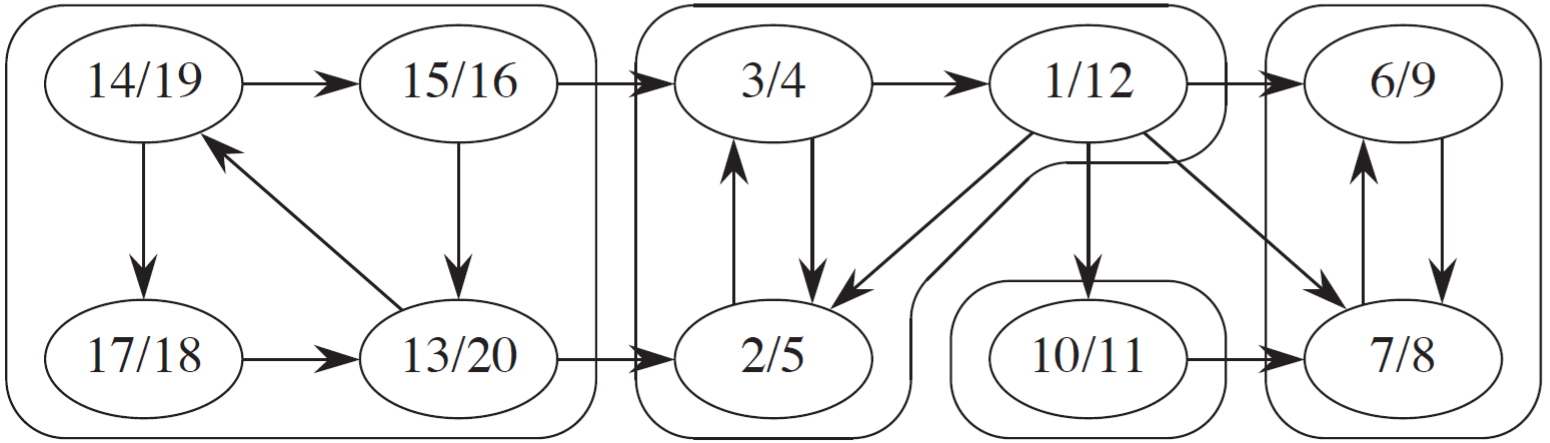
SCC(G)
  call DFS(G) to compute finish times $u.f$ for each vertex u
  create G^T
  call DFS(G^T), but in the main loop, consider vertices in order of decreasing $u.f$
      (as computed in first DFS)
  output the vertices in each tree of the depth-first forest formed in second DFS
      as a separate SCC

Do DFS in G.



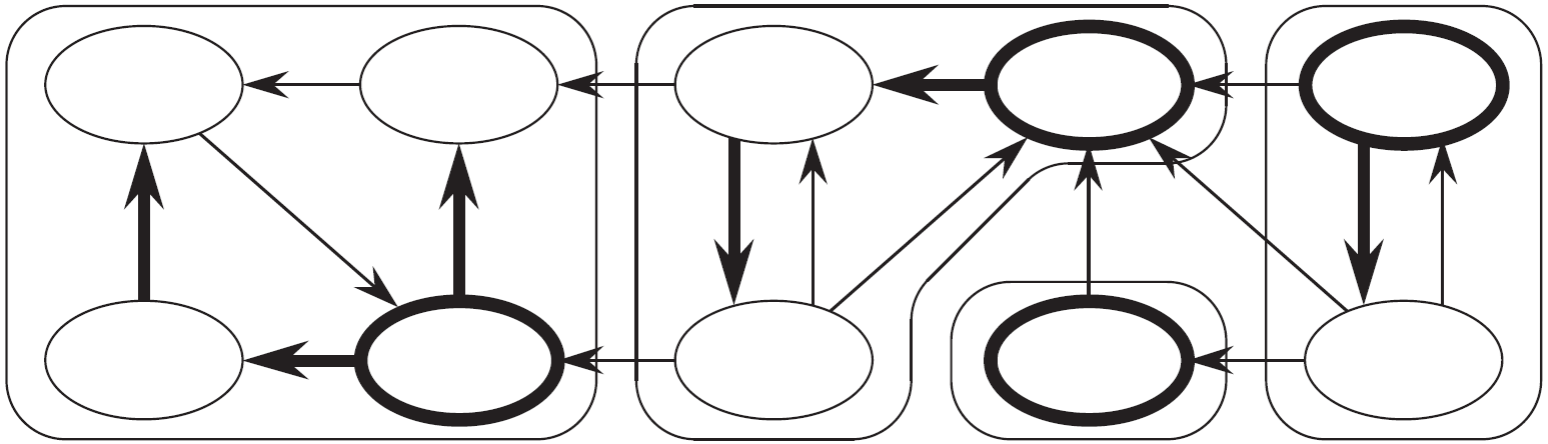
EXAMPLE (continued)

Run DFS.



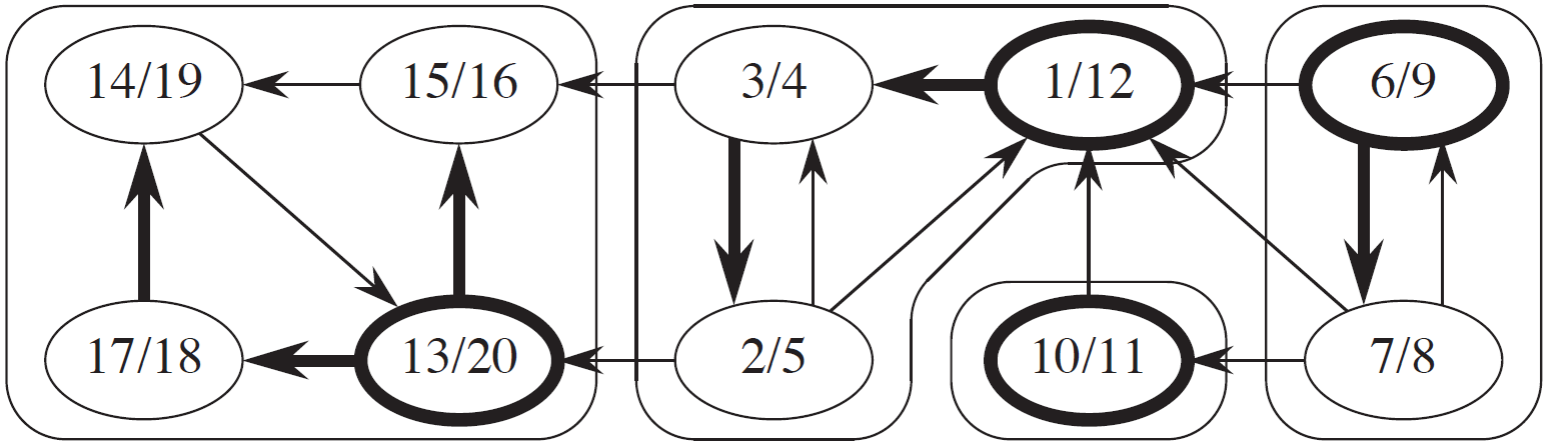
EXAMPLE (continued)

Take G^T.



EXAMPLE (continued)

DFS in G^T.



Time: $\Theta(V + E)$.

Each top level DFS-Visit will discover it's component.

EXAMPLE (continued)

How can this possibly work?

Idea

By considering vertices in second DFS in decreasing order of finish times from first DFS, visiting vertices of the component graph in topological sort order.

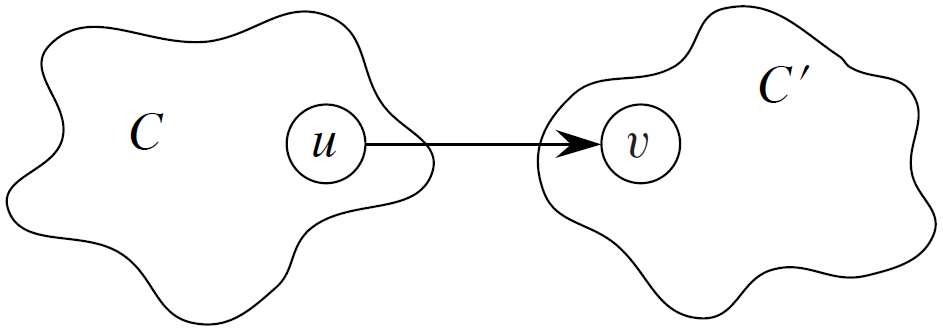
To prove that it works, first deal with 2 notational issues:

- Will be discussing $u.d$ and $u.f$. These always refer to *first* DFS.
- Extend notation for d and f to sets of vertices $U \subseteq V$:
  - $d(U) = \min\{u.d : u \in U\}$ (earliest discovery time in U)
  - $f(U) = \max\{u.f : u \in U\}$ (latest finish time in U)

EXAMPLE (continued)

Lemma

Let C and C' be distinct SCC's in $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$ such that $u \in C$ and $v \in C'$.



Then $f(C) > f(C')$.

STRONGLY CONNECTED COMPONENTS

Each root chosen for the second DFS can reach only

- vertices in its SCC—get tree edges to these,
- vertices in SCC's *already visited* in second DFS—get *no* tree edges to these.

Visiting vertices of $(G^{\mathrm{T}})^{\mathrm{SCC}}$ in reverse of topologically sorted order.

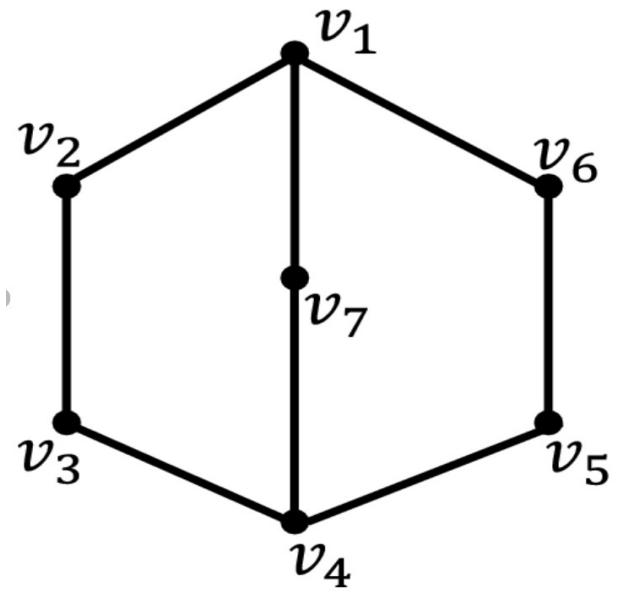
Applications-2

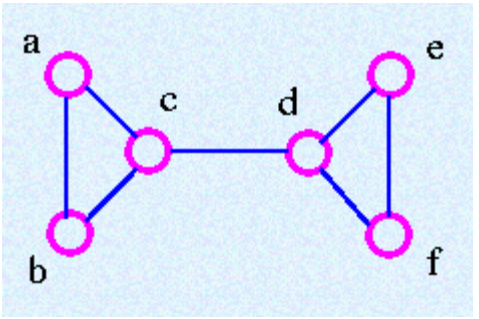
6. Check if the given graph G is strongly connected.
7. Given a directed graph G, output all the strongly connected components.
8. **Given a graph G, check if G is 2-edge-connected.**
9. Given a graph G, check if G is 2-vertex-connected.
10. Check if a graph G is planar.

2-edge-connected

NO Bridge edge

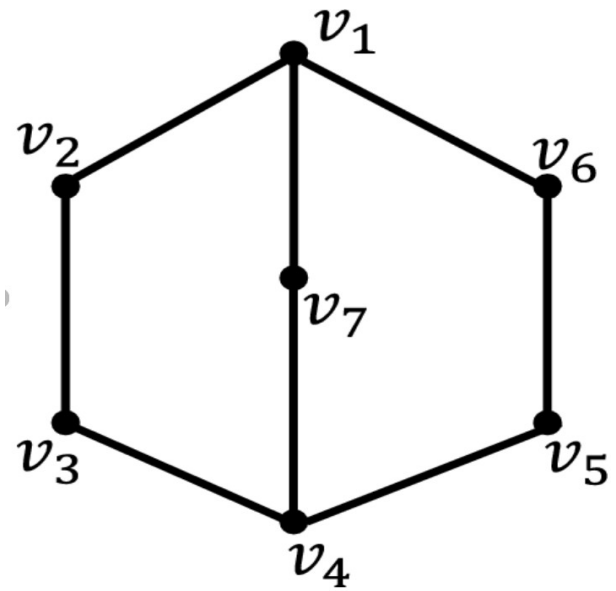
A graph G is **2-edge-connected** if removal of any edge does not disconnect the graph.





2-edge connected

There is no bridge edge.



Solution-1: Remove an edge and perform DFS/BFS to check for connectivity. Perform the above step for each edge. If it results in a disconnected graph, then the edge is a bridge edge, and say “NO”.

Time: O(E(V+E)).

2-edge connected

There is no bridge edge.

Solution-2:

1. Consider a vertex v and the subtree rooted at v, say T_v**.**

2. When we backtrack from v, there is a backedge to an ancestor of v, from some vertex in T_v**.**

3. What kind of backedges should we keep? The one that is discovered first.

2-edge connected

Solution-2:

1. Consider a vertex v and the subtree rooted at v, say T_v**.**

2. When we backtrack from v, there is a backedge to an ancestor of v, from some vertex in T_v**.**

3. **What kind of backedges should we keep? The one that is discovered first.**

How to approach step 3?

PSEUDOCODE (continued)

```
DFS-VISIT(G, u)
 1   time = time + 1
 2   u.d = time
 3   u.color = GRAY
 4   for each vertex v in G.Adj[u]
 5         if v.color == WHITE
 6               v.π = u
 7               DFS-VISIT(G, v)
 8   time = time + 1
 9   u.f = time
10   u.color = BLACK
```

Applications-2

5. Check if the given graph G is strongly connected.

6. Given a directed graph G, output all the strongly connected components.

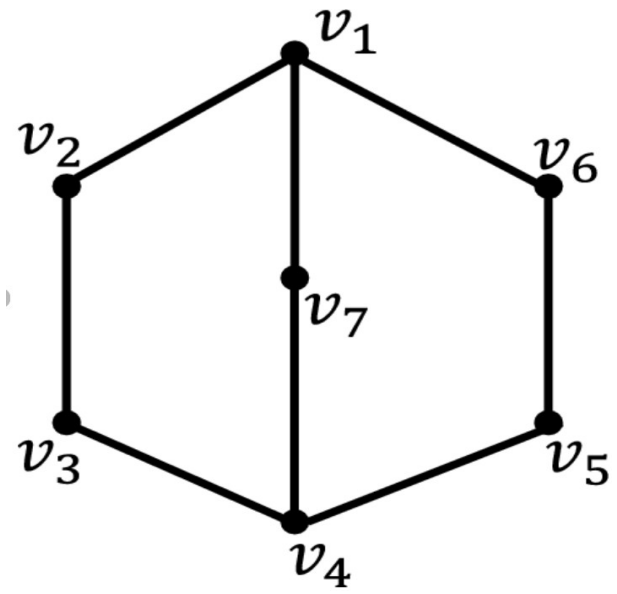
7. Given a graph G, check if G is 2-edge-connected.

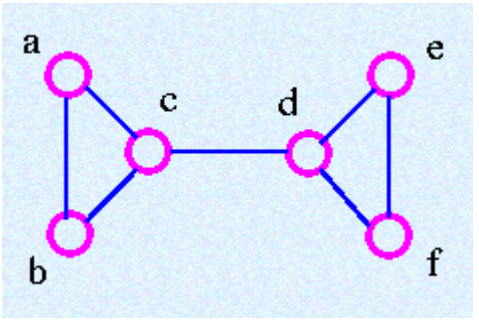
8. **Given a graph G, check if G is 2-vertex-connected.**

9. Check if a graph G is planar.

2-vertex connected

A graph G is **2-vertex connected** if removal of any vertex does not disconnect the graph.





Cut vertices

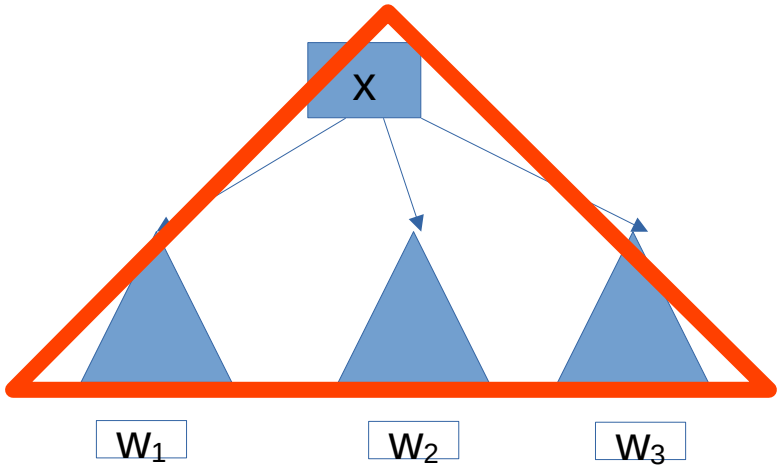
2-vertex connected

A graph that is

- 2-edge connected but 1-vertex connected?
- 2-vertex connected but 1-edge connected?

2-vertex connected: algorithm

At the node x, what condition should we check?



Applications of DFS in directed graphs

Topological sorting.

DAG.

Strongly connected graphs.

Applications of DFS in undirected graphs

2-edge connectivity

2-vertex connectivity

Applications of BFS in undirected graphs

Shortest distance between vertices.

Finding connected components of the graph.

Checking is a graph is bipartite.